*Trabajo Práctico Obligatorio: SkillSwap

Marketplace de talento, mentorías y proyectos · Backend con Java, Hibernate y JPA

Misión. Construir la lógica de una plataforma donde profesionales ofrecen mentorías o servicios breves, las organizaciones publican desafíos y los usuarios contratan usando créditos ficticios. Se entrega una interfaz Java con datos mock; el equipo debe conectarla con sus controllers y casos de uso.

1. Condiciones generales

- Se conformarán equipos de hasta **5 integrantes.**
- La solución será desarrollada en **Java** con **Hibernate/JPA** y **MySQL**. **No** se utilizará Spring Boot.
- El foco de evaluación es el **backend**: **dominio**, **controllers**, **casos de uso**, **arquitectura**, **persistencia** y **pruebas**.
- Cada entrega deberá incluir el **código fuente**, el **modelo UML** en **StarUML** y un informe **PDF** con las **decisiones de diseño**.
- En la defensa, cualquier integrante deberá poder explicar y modificar una parte de la solución.
- **No** se evaluará el diseño visual. *Se proveerá una interfaz Java de referencia con datos mockeados.*

2. Contexto del producto

SkillSwap nace dentro de una empresa que quiere conectar personas que necesitan resolver problemas concretos con profesionales dispuestos a enseñar o prestar un servicio. Un usuario puede actuar como cliente, proveedor de talento o ambas cosas. La plataforma combina una red profesional con un marketplace, pero todas las operaciones económicas se realizan con créditos ficticios llamados SkillCoins.

Los ejemplos incluyen una mentoría de preparación para entrevistas, una revisión de portfolio, una asesoría de arquitectura, una sesión de fotografía o la creación de un prototipo. El sistema deberá ser extensible para incorporar nuevas categorías, reglas de precio, medios de notificación y proveedores externos.

3. Objetivos de aprendizaje

Competencia	Evidencia esperada
Clean Architecture	Dependencias hacia el dominio; casos de uso independientes de la interfaz, JPA y proveedores externos.
SOLID y GRASP	Responsabilidades cohesivas, bajo acoplamiento, polimorfismo y controllers delgados.
Patrones de diseño	Patrones justificados por variaciones reales del problema, no por cantidad.
Persistencia	Modelo relacional consistente, transacciones, relaciones JPA y consultas eficientes.
Calidad	Pruebas automatizadas sobre reglas, controllers, casos de uso y persistencia.

4. Alcance funcional obligatorio

4.1 Perfiles y reputación

- Registrar un perfil profesional referenciando un userId provisto por un servicio de identidad externo.
- Administrar nombre visible, biografía, zona horaria, habilidades, intereses y disponibilidad.
- Consultar el perfil público con reputación, publicaciones activas e historial resumido.
- La reputación se recalcula a partir de calificaciones verificadas; no puede editarse manualmente.

4.2 Publicaciones

Un profesional podrá publicar una oferta de mentoría o servicio. Una organización podrá publicar un desafío. Toda publicación tendrá título, descripción, categoría, modalidad (remota/presencial), duración estimada, cupo, precio base en SkillCoins y etiquetas.

- **BORRADOR**: editable y no visible.
- **PUBLICADA**: visible y disponible para recibir solicitudes.
- **PAUSADA**: visible para su autor, pero no admite nuevas solicitudes.
- **CERRADA**: finalizada; no puede reabrirse.
- **CANCELADA**: retirada con motivo obligatorio.

Las transiciones inválidas deberán ser rechazadas por el dominio. Publicar requiere título, categoría, precio no negativo y al menos una franja de disponibilidad.

4.3 Búsqueda y recomendación

- Buscar publicaciones por texto, categoría, modalidad, rango de precio, habilidad y reputación mínima.
- Ordenar por relevancia, mejor reputación, menor precio o fecha más reciente.
- Implementar al menos dos estrategias de recomendación intercambiables: afinidad por habilidades y popularidad reciente.
- Paginar todos los listados devueltos por los controllers de consulta.

4.4 Solicitud, reserva y contratación

- Un cliente solicita una publicación indicando mensaje, franja elegida y objetivo.
- El proveedor acepta o rechaza. La aceptación crea una reserva y bloquea el cupo de manera transaccional.
- El cliente confirma el pago con SkillCoins. No se permiten saldos negativos ni doble cobro.
- Una reserva puede reprogramarse antes de comenzar si ambas partes aceptan.
- Al completar la sesión, se habilita una calificación por cada parte; una persona no puede calificarse a sí misma.

4.5 Estados de una contratación

Estado	Acciones válidas
PENDIENTE	Aceptar o rechazar.
ACEPTADA	Pagar, cancelar o proponer reprogramación.
CONFIRMADA	Iniciar, reprogramar o cancelar según política.
EN_CURSO	Completar.
COMPLETADA	Calificar y consultar comprobante.
RECHAZADA / CANCELADA	Solo consultar historial.

4.6 Notificaciones

Los eventos relevantes —solicitud recibida, aceptación, pago confirmado, reprogramación, cancelación y sesión completada— deberán notificar a las partes. Cada usuario elige email, push o notificación interna. Los proveedores reales se simulan, pero deben conectarse mediante interfaces reemplazables.

5. Reglas de negocio mínimas

- El autor no puede solicitar su propia publicación.
- Una publicación no publicada no admite solicitudes.
- No puede reservarse una franja ocupada ni superar el cupo.
- El débito y crédito de **SkillCoins** se realiza en una única transacción.
- La cancelación devuelve 100 %, 50 % o 0 % según la anticipación configurable.
- Las promociones pueden modificar el precio, pero nunca producir un total negativo.
- Solo participantes de una contratación completada pueden calificarla una vez.
- Toda transición de estado registra fecha, actor y motivo en un historial inmutable.
- La baja de un perfil no elimina operaciones históricas: se realiza baja lógica.
- Los controllers que modifican información deben validar permisos a partir del userId recibido del servicio de identidad mock.

6. Arquitectura obligatoria

La solución deberá respetar el principio de dependencias de Clean Architecture. Los nombres de paquetes son libres, pero se debe reconocer con claridad la siguiente separación:

Capa	Responsabilidad	No debería conocer
Dominio	Entidades, value objects, reglas, políticas y eventos.	Interfaz gráfica, JPA, MySQL.
Aplicación	Casos de uso, puertos de entrada/salida y coordinación transaccional.	Controllers concretos y SDK externos.
Infraestructura	JPA, repositorios, adaptadores externos, scheduler y configuración.	Decisiones de negocio nuevas.
Presentación	Interfaz Java, modelos de vista y captura de datos.	Detalles de persistencia y reglas de negocio.
Controllers	Reciben acciones de la interfaz, validan datos de entrada y delegan en casos de uso.	SQL, componentes visuales y reglas internas de entidades.

7. Patrones de diseño

Se deberán aplicar al menos cuatro patrones de diseño, además de evidenciar SOLID y GRASP. No es obligatorio usar todos. Cada patrón debe resolver una variación o dependencia concreta y estar respaldado por pruebas.

Patrón sugerido	Variación del problema
Strategy	Cálculo de recomendación, precio o política de cancelación.
State	Comportamiento y transiciones de publicación o contratación.
Observer	Publicación de eventos y notificaciones a múltiples interesados.
Adapter	Identidad, email, push o pasarela de SkillCoins simulada.
Simple Factory	Creación del canal de notificación o estrategia seleccionada.
Decorator	Promociones, comisiones o beneficios acumulables sobre el precio.
Facade	Vista resumida del perfil o coordinación simplificada del checkout.

Patrón sugerido	Variación del problema
Composite	Catálogo jerárquico de categorías y habilidades.
Singleton	Solo si se demuestra una única instancia necesaria y se justifican sus consecuencias.

8. Persistencia con MySQL, Hibernate y JPA

- Persistir perfiles, habilidades, publicaciones, franjas, solicitudes, contrataciones, movimientos de créditos, calificaciones, notificaciones e historial de estados.
- Modelar correctamente cardinalidades, claves, restricciones de unicidad e índices.
- Usar migraciones versionadas con Flyway o Liquibase; no depender de ddl-auto=create en la entrega.
- Definir límites transaccionales para aceptación, reserva, pago y cancelación.
- Evitar N+1 en consultas principales y justificar fetch plans o proyecciones.
- Incluir datos de demostración reproducibles para el entorno local.
- Implementar optimistic locking o una solución equivalente para evitar sobreventa de cupos.

9. Controllers mínimos

Los controllers serán clases Java normales. Recibirán comandos o parámetros desde la interfaz, invocarán los casos de uso y devolverán resultados simples. No se implementarán endpoints, HTTP ni una API REST.

Controller y operación	Propósito
PerfilController.crearPerfil(...)	Crear perfil profesional.
PerfilController.verPerfil(id)	Ver perfil y reputación.
PublicacionController.crearBorrador(...)	Crear publicación en borrador.
PublicacionController.cambiarEstado(...)	Ejecutar una transición de estado.
BusquedaController.buscar(filtros)	Buscar, filtrar, ordenar y paginar.
RecomendacionController.recomendar(usuario)	Obtener recomendaciones para el usuario.
ContratacionController.solicitar(...)	Solicitar una publicación.
ContratacionController.aceptar(id)	Aceptar y reservar cupo.
ContratacionController.pagar(id)	Confirmar pago con SkillCoins.
ContratacionController.cancelar(id)	Cancelar aplicando política de reintegro.
ContratacionController.completar(id)	Completar contratación.
CalificacionController.calificar(...)	Registrar calificación verificada.
NotificacionController.listar(usuario)	Listar notificaciones del usuario.

Los nombres son orientativos. Los controllers deben permanecer delgados, no contener SQL ni reglas de negocio y manejar los errores de aplicación de forma uniforme. Podrán utilizar command objects o modelos de entrada simples para evitar listas extensas de parámetros.

10. Interfaz provista: “Mock UI Kit”

La cátedra entregará una interfaz Java ya funcional, basada en Swing o JavaFX, que inicialmente utilizará un MockSkillSwapController con datos en memoria. Los equipos no deberán rediseñar las pantallas: deberán implementar los controllers reales y conectarlos mediante interfaces Java.

Incluido	Responsabilidad del equipo
Inicio, buscador y filtros	Invocar BusquedaController y RecomendacionController.
Detalle de publicación	Invocar solicitud y mostrar errores de aplicación.
Panel del proveedor	Invocar cambios de estado, aceptación y cupos.
Wallet de SkillCoins	Consultar saldo y movimientos mediante controller.
Notificaciones	Consultar bandeja y marcar lectura.
Perfiles y calificaciones	Consultar perfil, reputación e historial.

Contrato de integración. La interfaz y la lógica se ejecutarán dentro de la misma aplicación Java. La UI conocerá interfaces de controllers, nunca repositorios ni entidades JPA. Un login simulado entregará el userId activo a cada operación; no se implementará autenticación real.

11. Entregas incrementales

Hito	Contenido	Demostración
1 · Diseño	Alcance, historias, modelo de dominio, diagrama de clases, decisiones de arquitectura y patrones candidatos.	Recorrer un caso de uso sobre el UML.
2 · Núcleo	Dominio y casos de uso principales con puertos y pruebas unitarias.	Publicar, solicitar, aceptar y validar estados.
3 · Infraestructura	JPA/MySQL, repositories y adaptadores externos mock.	Persistencia real y flujo transaccional.
4 · Producto	Interfaz conectada a controllers, documentación, pruebas y versión final.	Demo completa y defensa técnica.

12. Pruebas y calidad

- Pruebas unitarias del dominio y de cada estrategia/política.
- Pruebas de casos de uso con puertos simulados.
- Pruebas de persistencia con una base compatible y datos controlados.
- Pruebas de integración de controllers, casos de uso y persistencia.
- Casos negativos: transición inválida, saldo insuficiente, cupo agotado, doble pago y calificación no autorizada.
- Cobertura mínima sugerida: 80 % en dominio y aplicación. La cobertura no reemplaza la calidad de los casos.
- README con instrucciones reproducibles para ejecutar la aplicación.

13. Consignas de entrega

- Diagrama de clases completo de dominio y aplicación en StarUML, con relaciones, multiplicidades y patrones identificados.
- Código fuente organizado y repositorio con el trabajo del equipo.
- Informe breve de decisiones: problema, alternativa elegida, consecuencias y evidencia en código/pruebas.

- Demostración en vivo del recorrido principal: publicar → solicitar → aceptar → pagar → completar → calificar.

14. Evaluación

Criterio	Se observa
Modelo y reglas de negocio	Invariantes, estados, value objects y comportamiento en el dominio.
Clean Architecture	Separación de capas, puertos, dependencias y casos de uso.
SOLID, GRASP y patrones	Responsabilidades, extensibilidad, justificación y pruebas.
Persistencia y transacciones	JPA, MySQL, migraciones, concurrencia y consultas.
Controllers e integración	Controllers delgados, manejo de errores e interfaz conectada.
Pruebas y calidad	Cobertura significativa, casos negativos y mantenibilidad.
Documentación y defensa	Trazabilidad, claridad y dominio individual de la solución.

15. Casos de aceptación obligatorios

1. Publicar una mentoría válida y encontrarla mediante búsqueda filtrada.
2. Rechazar la publicación de una oferta incompleta.
3. Solicitar, aceptar y pagar una mentoría con saldo suficiente.
4. Evitar que dos aceptaciones consuman el último cupo.
5. Rechazar un pago con saldo insuficiente sin modificar saldos ni estado.
6. Cancelar y calcular el reintegro con dos políticas diferentes.
7. Cambiar la estrategia de recomendación sin modificar el caso de uso consumidor.
8. Enviar dos tipos de notificación ante un mismo evento mediante adaptadores mock.
9. Aplicar promociones acumulables sin permitir precio negativo.
10. Completar una contratación, calificarla una sola vez y recalcular reputación.

16. Fuera de alcance

- Autenticación, registro, recuperación de contraseña y autorización OAuth/JWT real.
- Dinero real, integración bancaria o procesamiento de tarjetas.
- Chat en tiempo real, videollamadas y almacenamiento de archivos pesados.
- Despliegue productivo de alta disponibilidad.
- Diseño visual responsivo desde cero.z